

T.C. İSTANBUL RUMELİ ÜNİVERSİTESİ

Mühendislik Fakültesi — Bilgisayar Mühendisliği

BLG462 — BÜYÜK VERİ DÖNEM PROJESİ

Twitter Verisi ile Uçtan Uca Büyük Veri Pipeline'ı Proje Raporu

Ad Soyad	Berat Turan
Öğrenci Numarası	221201018
Ders	BLG462 — Büyük Veri
Dönem	Bahar Dönemi 2025-2026
Github Repo	https://github.com/BeratTrn/twitter_big_data_pipeline
Teslim Tarihi	8 Haziran 2026

İÇİNDEKİLER

İÇİNDEKİLER	1
GİRİŞ VE PROJE AMACI	2
AŞAMA 1: VERİ TEMİNİ VE KEŞİF	3
1.1. Veri Seti Hakkında	3
1.2. DIKW Piramidi Çerçevesinde Veri Setinin Değerlendirilmesi	3
1.3. Veri Keşfi ve Temel İstatistikler	3
AŞAMA 2: VERİ TEMİZLEME VE HAZIRLIK	5
2.1. Null Analizi ve Strateji Belirleme	5
2.2. Temizleme İşlemleri	6
2.3. Veri Kalitesi Değerlendirmesi	8
AŞAMA 3: MONGODB'YE YÜKLEME VE NOSQL SORGULARI	9
3.1. MongoDB Şema Tasarımı	9
3.2. Veri Yükleme (Bulk Insert)	9
3.3. Sorgular (Toplam 12 İşlem)	10
Sorgu 1 — Çoklu Koşul + Projeksiyon + Sıralama	10
Sorgu 2 — \$or + \$gt Filtreleme	11
Sorgu 3 — \$ne + \$exists + Sayfalama	11
Sorgu 4 — Çoklu İstatistik (\$group içinde \$sum + \$avg + \$min + \$max)	12
Sorgu 5 — Koşullu Etiketleme (\$match + \$group + \$project + \$cond)	12
Sorgu 6 — Yüzdesel Dağılım (\$addFields + \$divide + \$multiply)	13
Sorgu 7 — Bileşik İndeks (Compound Index) + explain()	14
Sorgu 8 — updateOne() + \$set ile Çoklu Alan Ekleme	15
Sorgu 9 — updateMany() + \$set + \$inc Birlikte Kullanım	15
Sorgu 10 — deleteMany() + Çoklu Koşul	16
Sorgu 11 — Regex + \$in Birleşik Kullanım	16
Sorgu 12 — \$in + Aggregation Pipeline (Bileşik _id)	17
Sorgu 13 — \$facet ile Çok Boyutlu Analiz	19
Sorgu 14 — \$bucket ile Güven Skoru Aralık Analizi	20
Sorgu 15 — distinct() ile Benzersiz Değer Analizi	21
AŞAMA 4: PYSPARK İLE DAĞITIK VERİ İŞLEME	22
4.1. Dönüşüm (Transformation) İşlemleri — 5 adet	23
İşlem 1 — filter + select (Karmaşık Filtreleme)	23
İşlem 2 — withColumn + when/otherwise (Koşullu Sütun)	24
İşlem 3 — groupBy + Çoklu Aggregation	24
İşlem 4 — UDF (User Defined Function) ile Tweet Uzunluk Kategorisi	25
İşlem 5 — Window Function ile Sıralama	25

4.2. Eylem (Action) İşlemleri — 2 adet	26
İşlem 6 — count + distinct (Farklı Sayım Türleri)	27
İşlem 7 — describe (İstatistiksel Özet)	27
4.3. Spark SQL — 2 adet	27
İşlem 8 — CASE WHEN + GROUP BY + HAVING	27
İşlem 9 — Subquery (İç İç Sorgu)	28
4.4. Pandas vs PySpark Karşılaştırması — 1 adet	29
İşlem 10 — Aynı İşlemi Pandas ve PySpark ile Yapıp Süre Karşılaştırması	30
ASAMA 5: VERİ GÖRSELLEŞTİRME VE ANALİZ	30
5.1. Duygu Dağılımı (Pie Chart)	31
5.2. Kelime Bulutu (WordCloud)	31
5.3. Zaman Serisi (Line Plot)	32
5.4. En Aktif 10 Kullanıcı (Bar Plot)	33
5.5. Şikayetlerin Kök Nedenleri (Horizontal Bar)	34
5.6. Havayolu Bazlı Duygu Karşılaştırması (Stacked Bar Chart)	35
5.7. Şikayet Sebebi - Havayolu Isısı Haritası	36
AŞAMA 6: ETL DİYAGRAMI VE SİSTEM MİMARİSİ	37
6.1. ETL vs. ELT Karşılaştırması	38
6.2. Uçtan Uca Proje Akış Diyagramı	38
SONUÇ	39

GİRİŞ VE PROJE AMACI

Bu projede Kaggle'dan indirdiğim "Twitter US Airline Sentiment" veri setini kullanarak baştan sona bir büyük veri pipeline'ı kurdum. Temel olarak şunları yaptım: ham tweet verisini Pandas ile temizledim, temiz veriyi MongoDB'ye yükleyip çeşitli sorgular yazdım, PySpark ile dağıtık işleme mantığını uyguladım ve son olarak veriden anlamlı grafikler çıkardım. Raporun geri kalanında bu adımların her birini kodlarıyla ve çıktılarıyla birlikte anlatıyorum.

AŞAMA 1: VERİ TEMİNİ VE KEŞİF

1.1. Veri Seti Hakkında

Kaggle'dan indirdiğim veri seti, ABD'deki havayolu şirketlerine atılmış yaklaşık 14.000 tweetten oluşuyor. Toplamda 15 sütun var: tweet metni, hangi havayoluna atıldığı, duygu durumu (pozitif/negatif/nötr), negatifse sebebi, güven skoru, kullanıcı adı, konum, tarih gibi bilgiler. Veri seti zaten etiketlenmiş durumda, yani her tweetin duygu durumu önceden belirlenmiş.

1.2. DIKW Piramidi Çerçevesinde Veri Setinin Değerlendirilmesi

Bu veri setini DIKW piramidine oturtmaya çalıştığımda şöyle bir tablo çıktı:

Data (Veri): Elimdeki ham hali 14.640 satırlık bir CSV dosyası. İçinde tweet metinleri, tarihler, havayolu isimleri var ama düzenlenmemiş, temizlenmemiş bir yığın. Bu aşamada veri sadece karakter ve sayılardan ibaret; herhangi bir anlam çıkarımı yapılmamış durumda. Örneğin "570306133677760513, neutral, 1.0, NaN, Virgin America, @VirginAmerica What @dhepburn said." şeklinde bir satır tek başına bir şey ifade etmiyor.

Information (Bilgi): Bu ham veriye baktığımda tweetlerin pozitif, negatif ve nötr olarak etiketlendiğini görüyorum. Negatif olanların da "rötar", "kayıp bagaj", "müşteri hizmetleri" gibi alt kategorilere ayrıldığını fark ediyorum. Yani artık sadece sayı değil, bağlamsal bir anlam var. Veri, belirli bir soru çerçevesinde yapılandırılmış oluyor. Örneğin "bu tweette müşteri memnun mu?" sorusuna artık cevap verebiliyorum.

Knowledge (Bilgi Birikimi): Veriyi analiz ettiğimde mesela "United Airlines en çok müşteri hizmetleri konusunda şikayet alıyor" ya da "şikayetler belirli günlerde patlama yapıyor" gibi örüntüler ortaya çıkıyor. Bu artık karar vermeye yakın bir seviye. Birden fazla bilgi parçası birleşiyor ve anlamlı bir örüntü ortaya çıkıyor. Farklı havayollarının farklı sorunları olduğunu, belirli zaman dilimlerinde kriz yaşandığını anlıyorum.

Wisdom (Bilgelik): Piramidin en tepesi. Burası artık "peki ne yapalım?" sorusunun cevabı. Mesela bir havayolu şirketi bu verilere bakarak müşteri hizmetleri ekibini güçlendirebilir ya da bagaj takip sistemini yenileyebilir. Yani veriden stratejik karara dönüşüm. Bilgi birikimi, deneyim ve bağlamla birleştiğinde eyleme dönüşen iç görüleri ulaşıyor. Örneğin "müşteri hizmetleri şikayet oranını %20 azaltmak için çağrı merkezi kapasitesini kriz günlerinde ikiye katla" gibi somut bir aksiyon planı çıkarılabilir.

1.3. Veri Keşfi ve Temel İstatistikler

Pandas ile veri setini okuttuğumda 14640 satır ve 15 sütun olduğunu gördüm. `df.info()` ile baktığımda bazı sütunlarda ciddi miktarda eksik değer olduğunu fark ettim (özellikle `negativereason`, `airline_sentiment_gold`, `tweet_coord`). `df.describe()` çıktısı da sayısal sütunların dağılımını gösterdi.

```
import pandas as pd
import re

# veri setini okuyorum, encoding latin-1 olmak zorunda yoksa turkce karakterlerde patlıyor
df = pd.read_csv('../data/raw/Tweets.csv', encoding='latin-1')

# boyutuna bakalım kac satir kac sutun var
print("Satir x Sütün:", df.shape)
print("\nSütün isimleri:\n", df.columns.tolist())

# ilk 5 kayda goz atalım
df.head()
```

Satir x Sütün: (14640, 15)

Sütün isimleri:

['tweet_id', 'airline_sentiment', 'airline_sentiment_confidence', 'negativereason', 'negativereason_confidence', 'airline', 'airline_sentiment_gold', 'name', 'negative_reason', 'tweet_coord', 'tweet_created', 'tweet_location', 'user_timezone', 'user_location']

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name	negative_reason
0	570306133677760513	neutral	1.0000	NaN	NaN	Virgin America	NaN	cairdin	
1	570301130888122368	positive	0.3486	NaN	0.0000	Virgin America	NaN	jnardino	
2	570301083672813571	neutral	0.6837	NaN	NaN	Virgin America	NaN	yvonnalynn	
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Virgin America	NaN	jnardino	
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Virgin America	NaN	jnardino	

Şekil 1: `df.shape` ve `df.head()` çıktısını açıklıyor — veri boyutu ve NaN yoğunluğuna dikkat çekiyor

```
print("--- Veri Tipleri ve Eksik Değer Durumu ---")
df.info()

print("\n--- Temel İstatistikler ---")
df.describe(include='all')
```

0.1s

```
--- Veri Tipleri ve Eksik Değer Durumu ---
<class 'pandas.DataFrame'>
RangeIndex: 14640 entries, 0 to 14639
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tweet_id                             14640 non-null  int64
1   airline_sentiment                    14640 non-null  str
2   airline_sentiment_confidence         14640 non-null  float64
3   negativereason                       9178 non-null   str
4   negativereason_confidence            10522 non-null  float64
5   airline                              14640 non-null  str
6   airline_sentiment_gold                40 non-null     str
7   name                                 14640 non-null  str
8   negativereason_gold                   32 non-null     str
9   retweet_count                        14640 non-null  int64
10  text                                 14640 non-null  str
11  tweet_coord                          1019 non-null   str
12  tweet_created                        14640 non-null  str
13  tweet_location                       9907 non-null   str
14  user_timezone                        9820 non-null   str
dtypes: float64(2), int64(2), str(11)
memory usage: 1.7 MB
```

--- Temel İstatistikler ---

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name	negativereason
count	1.464000e+04	14640	14640.000000	9178	10522.000000	14640	40	14640	
unique	NaN	3	NaN	10	NaN	6	3	7701	
top	NaN	negative	NaN	Customer Service Issue	NaN	United	negative	JetBlueNews	Custom
freq	NaN	9178	NaN	2910	NaN	3822	32	63	
mean	5.692184e+17	NaN	0.900169	NaN	0.638298	NaN	NaN	NaN	
std	7.791112e+14	NaN	0.162830	NaN	0.330440	NaN	NaN	NaN	
min	5.675883e+17	NaN	0.335000	NaN	0.000000	NaN	NaN	NaN	
25%	5.685592e+17	NaN	0.692300	NaN	0.360600	NaN	NaN	NaN	
50%	5.694779e+17	NaN	1.000000	NaN	0.670600	NaN	NaN	NaN	
75%	5.698905e+17	NaN	1.000000	NaN	1.000000	NaN	NaN	NaN	
max	5.703106e+17	NaN	1.000000	NaN	1.000000	NaN	NaN	NaN	

Şekil 2-3: `df.info()` + `df.describe()` çıktısını açıklıyor — veri tipleri, eksik değerler ve temel istatistikleri özetliyor

AŞAMA 2: VERİ TEMİZLEME VE HAZIRLIK

2.1. Null Analizi ve Strateji Belirleme

Veri temizlemeye başlamadan önce her sütundaki eksik değer oranlarını inceledim. Bu adım çok önemli çünkü hangi sütunu sileceğime, hangisini dolduracağıma bu oranlara bakarak karar veriyorum.

```
# her sutundaki null oranini yuzde olarak hesapliyorum
# isnull() her hucreyi True/False yapar, sum() True lari toplar
# len(df) ye bolerek orana ceviriyorum
print("Toplam kayıt:", len(df))
print("\nNull oranlari (%):")
print((df.isnull().sum() / len(df) * 100).round(2))
```

✓ 0.0s

Toplam kayıt: 14640

Null oranlari (%):

tweet_id	0.00
airline_sentiment	0.00
airline_sentiment_confidence	0.00
negativereason	37.31
negativereason_confidence	28.13
airline	0.00
airline_sentiment_gold	99.73
name	0.00
negativereason_gold	99.78
retweet_count	0.00
text	0.00
tweet_coord	93.04
tweet_created	0.00
tweet_location	32.33
user_timezone	32.92
dtype: float64	

Şekil 4: Hangi sütunların ne kadar boş olduğunu vurgular

Bu oranları incelediğimde sütunları iki kategoriye ayırdım:

Kategori 1 — Düşürülecek Sütunlar (Drop):

airline_sentiment_gold (%99.73), negativereason_gold (%99.78) ve tweet_coord (%93.04) sütunları neredeyse tamamen boş. 14.640 kayıttan sadece 40 tanesi airline_sentiment_gold dolu, 32 tanesi negativereason_gold dolu, 1019 tanesi tweet_coord dolu. Bu sütunları veri setinde tutmanın hiçbir anlamı yok, istatistiksel olarak anlamlı bir analiz yapılamaz. Gereksiz boyut artışı yaratır daha fazla sütun demek daha karmaşık hesaplama demek. Modelleme aşamasında gürültü kaynağı olur %99 boş bir sütun analizi yanıltır. Veri biliminde genel kabul görmüş kurala göre, bir sütundaki eksik değer oranı %70'i geçerse o sütun bilgi taşıma kapasitesini yitirmiş sayılır. Bu yüzden bu üç sütunu tamamen düşürdüm.

Kategori 2 — Doldurulacak Sütunlar (Fillna):

negativereason (%37.31), tweet_location (%32.33) ve user_timezone (%32.92) sütunlarında eksiklik oranı %30-40 arasında. Bu sütunlar analizde kullanılabilir bilgi taşıyor. negativereason sadece negatif tweetlerde dolu, pozitif ve nötr tweetlerde doğal olarak boş. tweet_location kullanıcının profilinde konum bilgisi yoksa boş. user_timezone aynı şekilde, profilde timezone bilgisi yoksa boş. Satır silmek yerine bu boşlukları anlamlı placeholder değerlerle ("Belirtilmedi", "Bilinmiyor") doldurmak, veri bütünlüğünü korurken analiz bütünlüğünü bozmaz.

2.2. Temizleme İşlemleri

```
# temizleme oncesi kayit sayisini not aliyorum, sonra karsilastirmak icin
baslangic_kayit = len(df)
print(f"Temizleme oncesi toplam kayit: {baslangic_kayit}")

# --- ADIM 1: Yuksek null oranli sutunlari dusur (Boyut Azaltma) ---
# %90 uzeri bos olan sutunlar analize katkı saglamiyor
# bunlari tamamen cikariyorum, Boylece gereksiz boyuttan kurtuluyorum
dusurulecek_sutunlar = ['airline_sentiment_gold', 'negativereason_gold', 'tweet_coord']
df = df.drop(columns=dusurulecek_sutunlar)
print(f"\nDusurulen sutunlar: {dusurulecek_sutunlar}")
print(f"Kalan sutun sayisi: {len(df.columns)}")
```

✓ 0.0s

Temizleme oncesi toplam kayit: 14640

Dusurulen sutunlar: ['airline_sentiment_gold', 'negativereason_gold', 'tweet_coord']
Kalan sutun sayisi: 12

Şekil 5: 15'ten 12 sütuna düşme işlemini özetler

`df.drop(columns=[...])` ile 3 sütunu veri setinden tamamen çıkardım. 15 sütundan 12'ye düştük. Bu 3 sütun %90+ boş olduğu için analiz için kullanışsız. Bunları tutmak sadece gereksiz bellek kullanımı ve hesaplama karmaşıklığı yaratır.

```
# --- ADIM 2: Metin temizleme ---
# tweet metinlerinde URL, @mention, özel karakter gibi seyler var
# bunlar kelime analizini ve WordCloud'u bozar, temizlemem lazim
def metin_temizle(text):
    """tweet metnini temizleyen fonksiyon"""
    if type(text) != str:
        return text

    # url leri sil - http ile baslayan herseyi yakala ve sil
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)
    # @mention leri sil - @ isareti ve ardindan gelen kelimeyi sil
    text = re.sub(r'@\w+', '', text)
    # hashtag isaretini sil ama kelimeyi birak
    # ornegin #flight -> flight olur (kelime analiz icin ise yarayabilir)
    text = re.sub(r'#', '', text)
    # geriye kalan özel karakterleri temizle, sadece harf rakam bosluk kalsin
    text = re.sub(r'^\w\s', '', text)
    # fazla bosluklari duzelt ve kucuk harfe cevir
    text = " ".join(text.split()).lower()
    return text

# fonksiyonu text sutununa uygulayip yeni sutun olusturuyorum
# apply() her satira tek tek fonksiyonu uygular
df['cleaned_text'] = df['text'].apply(metin_temizle)

# temizlenmis haliyle eski halini yan yana gorelim
df[['text', 'cleaned_text']].head()
```


	text	cleaned_text
0	@VirginAmerica What @dhepburn said.	what said
1	@VirginAmerica plus you've added commercials t...	plus youve added commercials to the experience...
2	@VirginAmerica I didn't today... Must mean I n...	i didnt today must mean i need to take another...
3	@VirginAmerica it's really aggressive to blast...	its really aggressive to blast obnoxious enter...
4	@VirginAmerica and it's a really big bad thing...	and its a really big bad thing about it

Şekil 6-7: text vs cleaned_text karşılaştırmasını açıklar

Regex kullanarak tweet metinlerindeki URL, @mention ve özel karakterleri sildim. Temiz metni cleaned_text adında yeni bir sütuna yazdım. Örneğin "@VirginAmerica What @dhepburn said. <http://t.co/abc>" gibi bir tweet, temizlenmezse WordCloud'da "@VirginAmerica" en büyük kelime olarak çıkar — ama bu bir mention, anlamlı bir kelime değil. Temizledikten sonra sadece anlamlı kelimeler kalıyor.

```
# --- ADIM 3: Tekrar eden satirlari sil ---
# ayni tweet birden fazla kayıt olmus olabilir, bunlar analizi bozar
onceki = len(df)
df = df.drop_duplicates()
print(f"Silinen tekrarlı kayıt: {onceki - len(df)}")

# --- ADIM 4: Orta seviye null sutunlari anlamlı degerlerle doldur ---
# negativereason: pozitif/notr tweetlerde dogal olarak bos
# silmek yerine "Belirtilmedi" yaziyorum ki satirlari kaybetmeyeyim
df['negativereason'] = df['negativereason'].fillna('Belirtilmedi')

# negativereason_confidence: neden yoksa guven skoru da yok
# sayisal sutun olduğu için 0 ile dolduruyorum
df['negativereason_confidence'] = df['negativereason_confidence'].fillna(0)

# tweet_location: konum bilgisi vermemis kullanicilar
df['tweet_location'] = df['tweet_location'].fillna('Bilinmiyor')

# user_timezone: timezone bilgisi olmayan kullanicilar
df['user_timezone'] = df['user_timezone'].fillna('Bilinmiyor')

# --- ADIM 5: Veri tipi donusumleri ---
# tweet_created sutunu string olarak geliyor, datetime a ceviriyorum
# bu sayede sonra zaman serisi analizi yapabilecegim (gun bazlı gruplama vs.)
df['tweet_created'] = pd.to_datetime(df['tweet_created'])

# sonucu raporlayalım
bitis_kayit = len(df)
print(f"\nTemizleme öncesi kayıt: {baslangic_kayit}")
print(f"Temizleme sonrası kayıt: {bitis_kayit}")
print(f"Silinen tekrarlı kayıt: {baslangic_kayit - bitis_kayit}")
print(f"Dusurulen sutun sayisi: {len(dusurulecek_sutunlar)}")
print(f"Kalan sutun sayisi: {len(df.columns)}")

# son null kontrolü - artık hiçbir sutunda null olmamali
print("\nTemizleme sonrası null durumu:")
print(df.isnull().sum())
```

Silinen tekrarlı kayıt: 39

Temizleme öncesi kayıt: 14640

Temizleme sonrası kayıt: 14601

Silinen tekrarlı kayıt: 39

Dusurulen sutun sayisi: 3

Kalan sutun sayisi: 13

Temizleme sonrası null durumu:

```
tweet_id      0
airline_sentiment      0
airline_sentiment_confidence      0
negativereason      0
negativereason_confidence      0
airline      0
name      0
retweet_count      0
text      0
tweet_created      0
tweet_location      0
user_timezone      0
cleaned_text      0
dtype: int64
```

Şekil 8-9: temizleme sürecinin sonucunu özetliyor

2.3. Veri Kalitesi Değerlendirmesi

Derste öğrendiğimiz veri kalitesi boyutlarından üçünü bu projede doğrudan uyguladım.

Tamlık: Veri setinde özellikle negativereason, tweet_location ve user_timezone sütunlarında %30-37 arası boşluk vardı. Bunları silseydim binlerce satır kaybederdim. Onun yerine boş değerleri "Belirtilmedi" veya "Bilinmiyor" gibi anlamlı placeholder'larla doldurdum. %90 üzeri boş olan 3 sütunu ise veri setinden çıkararak boyut azaltma yaptım. Sonuçta hiçbir satır kaybetmeden veri tamlığı sağlanmış oldu.

Tutarlılık: Aynı tweeti birden fazla gösteren tekrarlı kayıtlar vardı (36 adet). Bunlar analizde sayıları şişirirdi. Mesela bir tweet 2 kere kayıtlıysa o hava yolunun tweet sayısı olduğu gibi 1 fazla gözükür. drop_duplicates() ile bunları temizledim ve veri setindeki her kaydın tekil olmasını sağladım.

Geçerlilik: tweet_created sütunu metin olarak geliyordu. Bu haliyle tarih bazlı filtreleme veya sıralama yapamazdım. Python "2015-02-24" stringini bir tarih olarak anlamaz, sadece karakter dizisi olarak görür. Bunu pd.to_datetime() ile datetime objesine çevirerek geçerli bir formata dönüştürdüm. Ayrıca metin temizleme ile tweetlerdeki URL, mention ve özel karakter kirliliğini giderdim.

AŞAMA 3: MONGODB'YE YÜKLEME VE NOSQL SORGULARI

3.1. MongoDB Şema Tasarımı

Projede SQL yerine MongoDB kullandığım için verileri düz doküman şeklinde modelledim. Her tweet tek bir BSON dokümanı olarak tutuldu. Tweet metni, havayolu, tarih, duygu durumu hepsi aynı dokümanın içinde. Neden bu yapıyı seçtim, tablolar arası join gerektiren bir ilişki yok her tweet bağımsız bir birim.

Flat document yapısı okuma performansını artırıyor çünkü tek bir sorguyla tüm bilgiye ulaşıyorum. NoSQL'in güçlü taraflarından biri şemasının esnek olması ileride yeni alan eklemem gerekirse tüm tabloyu değiştirmeme gerek yok.

3.2. Veri Yükleme (Bulk Insert)

Temizlenen veriyi PyMongo ile MongoDB'ye yükledim. Pandas DataFrame'i önce dict listesine çevirip insert_many() ile toplu yükleme (bulk insert) yaptım. Bu sayede 14.000+ doküman tek seferde veritabanına girdi.

```

import pandas as pd
from pymongo import MongoClient

# temizlenmis csv yi okuyorum
df = pd.read_csv('../data/processed/cleaned_tweets.csv')

# ONEMLI: Pandas NaN degerlerini Python None'a ceviriyorum
# boylece MongoDB'de duzgün "null" olarak saklanir
# bunu yapmazsam MongoDB'de "nan" string olarak kaydedilir ve sorgularda sorun cikarir
df = df.where(df.notna(), None)

# mongodb ye baglanma
client = MongoClient('mongodb://localhost:27017/')
print("MongoDB baglantisi basarili!")

# veritabani ve koleksiyon olustur
db = client['twitter_db_pipeline']
collection = db['tweetler']

# eger onceden veri varsa temizleyelim ki cift kayit olmasin
collection.delete_many({})

# pandas dataframe i dict listesine cevrip toplu yukleme (bulk insert)
# to_dict('records') her satiri bir sozluk (dict) yapar
# insert_many() hepsini tek seferde MongoDB'ye yazar – tek tek yazmaktan cok daha hizli
records = df.to_dict('records')
collection.insert_many(records)
print(f"{len(records)} doküman MongoDB'ye yuklendi.")
✓ 3.7s
MongoDB baglantisi basarili!
14601 doküman MongoDB'ye yuklendi.

```

Şekil 10: MongoDB bağlantısı ve 14.601 dokümanın bulk insert ile yüklenmesi

3.3. Sorgular (Toplam 12 İşlem)

Yönergedeki tabloya uygun olarak 15 farklı MongoDB işlemi gerçekleştirdim.

Sorgu 1 — Çoklu Koşul + Projeksiyon + Sıralama

\$and operatörü ile 3 koşulu aynı anda sağlayanlar: Virgin America + pozitif + güven skoru 1.0. Projeksiyon ile sadece ihtiyacım olan alanları getiriyorum (gereksiz veri transferini önler). Retweet sayısına göre azalan sıralama.

```

# $and: tum kosullarin AYNI ANDA saglanmasini ister
# projeksiyon (2. parametre): hangi alanlari getir hangileri getirme
# _id: 0 diyerek MongoDB'nin otomatik ekledigi _id alanini gizliyorum
# sort("retweet_count", -1): -1 azalan siralama demek
# limit(3): en fazla 3 sonuc getir
print("Sorgu 1: Virgin America + pozitif + guven=1.0:")
sonuc1 = collection.find(
    {"$and": [
        {"airline": "Virgin America"},
        {"airline_sentiment": "positive"},
        {"airline_sentiment_confidence": 1.0}
    ]},
    {"text": 1, "airline_sentiment_confidence": 1, "_id": 0}
).sort("retweet_count", -1).limit(3)
for doc in sonuc1:
    print(doc)

```

```
Sorgu 1: Virgin America + pozitif + guven=1.0:
{'airline_sentiment_confidence': 1.0, 'text': '@VirginAmerica Flying LAX to SFO and after looking at the awesome movie lineup I actually wish I was
{'airline_sentiment_confidence': 1.0, 'text': "Always have it together!!! You're welcome! RT @VirginAmerica: @jessicajaymes You're so welcome."}
{'airline_sentiment_confidence': 1.0, 'text': '@VirginAmerica thanks for gate checking my baggage on your full flight dfw-lax 883 and giving me ear
```

Şekil 11-12: \$and + projeksiyon + retweet sıralaması ile Sorgu 1 sonuçları

Sorgu 2 — \$or + \$gt Filtreleme

\$or ile "ya bu ya şu" mantığı kurdum — ya rötat ya iptal yaşayan yolcular. Üstüne \$gt (greater than) ile güven skoru 0.8'den büyük olanları filtreledim.

```
# $or: kosullardan EN AZ BIRININ saglanmasi yeterli
# $gt: "greater than" yani "buyuktur" – 0.8'den buyuk olanlar gelir
print("Sorgu 2: Rotat VEYA iptal + guven > 0.8:")
sonuc2 = collection.find(
    {
        "$or": [
            {"negativereason": "Late Flight"},
            {"negativereason": "Cancelled Flight"}
        ],
        "airline_sentiment_confidence": {"$gt": 0.8}
    },
    {"airline": 1, "negativereason": 1, "airline_sentiment_confidence": 1, "_id": 0}
).limit(3)
for doc in sonuc2:
    print(doc)
```

✓ 0.0s

```
Sorgu 2: Rotat VEYA iptal + guven > 0.8:
{'airline_sentiment_confidence': 1.0, 'negativereason': 'Late Flight', 'airline': 'Virgin America'}
{'airline_sentiment_confidence': 1.0, 'negativereason': 'Cancelled Flight', 'airline': 'Virgin America'}
{'airline_sentiment_confidence': 1.0, 'negativereason': 'Late Flight', 'airline': 'Virgin America'}
```

Şekil 13: \$or + \$gt kombinasyonu ile Sorgu 2 filtreleme sonuçları

Sorgu 3 — \$ne + \$exists + Sayfalama

Konum bilgisi "Bilinmiyor" olmayan (yani gerçekten konum paylaşan) kullanıcıları getiriyorum. skip/limit ile sayfalama yapıyorum — büyük veri setlerinde tüm sonuçları tek seferde getirmek yerine sayfa sayfa getirmek performans için önemli.

```
# $ne: "not equal" yani "esit degil" — "Bilinmiyor" olanlari disarida birak
# $exists: True — bu alan dokumanda var mi kontrol et
# skip(10): ilk 10 sonucu atla
# limit(3): sonraki 3 tanesini getir (2. sayfa gibi dusun)
print("Sorgu 3: Konum paylasan kullanicilar (sayfalama):")
sonuc3 = collection.find(
    {
        "tweet_location": {"$ne": "Bilinmiyor", "$exists": True},
        "airline_sentiment": "negative"
    },
    {"name": 1, "tweet_location": 1, "airline": 1, "_id": 0}
).skip(10).limit(3)
for doc in sonuc3:
    print(doc)
```

✓ 0.0s

Sorgu 3: Konum paylasan kullanicilar (sayfalama):
{ 'airline': 'Virgin America', 'name': 'noelduan', 'tweet_location': 'SF â\x86\x94 NY' }
{ 'airline': 'Virgin America', 'name': 'gianagon', 'tweet_location': 'New York + Panama' }
{ 'airline': 'Virgin America', 'name': 'seimatrui', 'tweet_location': 'Los Angeles' }

Şekil 14: \$ne + \$exists + sayfalama ile gerçek konum paylaşan kullanıcıların filtelenmesi

Sorgu 4 — Çoklu İstatistik (\$group içinde \$sum + \$avg + \$min + \$max)

Tek bir aggregation pipeline içinde her havayolu için 4 farklı istatistiği aynı anda hesaplıyorum. Bunu 4 ayrı sorgu ile de yapabilirdim ama pipeline ile tek geçişte yapmak çok daha verimli.

```
# $group: belirtilen alana gore grupla (SQL'deki GROUP BY gibi)
# _id: "$airline" — airline alanina gore grupla
# $sum: 1 — her dokuman icin 1 ekle (yani say)
# $avg: ortalama hesapla
# $min: en kucuk degeri bul
# $max: en buyuk degeri bul
# $sort: toplam_tweet'e gore azalan sirala
print("Sorgu 4: Havayolu bazli istatistikler:")
pipeline1 = [
    {"$group": {
        "_id": "$airline",
        "toplam_tweet": {"$sum": 1},
        "ort_guven": {"$avg": "$airline_sentiment_confidence"},
        "min_guven": {"$min": "$airline_sentiment_confidence"},
        "max_rt": {"$max": "$retweet_count"}
    }},
    {"$sort": {"toplam_tweet": -1}}
]
for doc in collection.aggregate(pipeline1):
    print(doc)
```

Sorgu 4: Havayolu bazli istatistikler:
{ '_id': 'United', 'toplam_tweet': 3822, 'ort_guven': 0.9008776818419676, 'min_guven': 0.335, 'max_rt': 7 }
{ '_id': 'US Airways', 'toplam_tweet': 2913, 'ort_guven': 0.9215784414692757, 'min_guven': 0.34, 'max_rt': 44 }
{ '_id': 'American', 'toplam_tweet': 2720, 'ort_guven': 0.916166911764706, 'min_guven': 0.3367, 'max_rt': 5 }
{ '_id': 'Southwest', 'toplam_tweet': 2420, 'ort_guven': 0.8865159504132231, 'min_guven': 0.3353, 'max_rt': 22 }
{ '_id': 'Delta', 'toplam_tweet': 2222, 'ort_guven': 0.8698782628262827, 'min_guven': 0.3363, 'max_rt': 31 }
{ '_id': 'Virgin America', 'toplam_tweet': 504, 'ort_guven': 0.8760861111111111, 'min_guven': 0.3482, 'max_rt': 4 }

Şekil 15-16: Tek pipeline'da 4 istatistik (\$sum, \$avg, \$min, \$max) — United en fazla, Virgin America en az tweet ile

Sorgu 5 — Koşullu Etiketleme (\$match + \$group + \$project + \$cond)

Önce negatif tweetleri filtreleyip havayoluna göre grupluyorum. Sonra \$cond ile her havayolunun ortalama güvenine bakıp "yüksek" veya "düşük" etiketi veriyorum. Bu, SQL'deki CASE WHEN yapısının MongoDB karşılığı.

```
# $match: filtreleme (SQL'deki WHERE gibi) – sadece negatif tweetler
# $group: havayoluna göre grupla, sayı ve ortalama hesapla
# $project: çıktı alanlarını belirle + yeni alan oluştur
# $cond: if/then/else mantığı – ort_guven >= 0.9 ise "yüksek" değilse "düşük"
# $round: ondalık basamak sayısını sınırla (3 basamak)
print("Sorgu 5: Negatif tweetlerde güven seviyesi etiketi:")
pipeline2 = [
    {"$match": {"airline_sentiment": "negative"}},
    {"$group": {
        "_id": "$airline",
        "negatif_sayisi": {"$sum": 1},
        "ort_guven": {"$avg": "$airline_sentiment_confidence"}
    }},
    {"$project": {
        "negatif_sayisi": 1,
        "ort_guven": {"$round": ["$ort_guven", 3]},
        "güven_seviyesi": {
            "$cond": {
                "if": {"$gte": ["$ort_guven", 0.9]},
                "then": "yüksek",
                "else": "düşük"
            }
        }
    }},
    {"$sort": {"negatif_sayisi": -1}}
]
for doc in collection.aggregate(pipeline2):
    print(doc)
```

Sorgu 5: Negatif tweetlerde güven seviyesi etiketi:

```
{'_id': 'United', 'negatif_sayisi': 2633, 'ort_guven': 0.933, 'güven_seviyesi': 'yüksek'}
{'_id': 'US Airways', 'negatif_sayisi': 2263, 'ort_guven': 0.946, 'güven_seviyesi': 'yüksek'}
{'_id': 'American', 'negatif_sayisi': 1939, 'ort_guven': 0.944, 'güven_seviyesi': 'yüksek'}
{'_id': 'Southwest', 'negatif_sayisi': 1186, 'ort_guven': 0.921, 'güven_seviyesi': 'yüksek'}
{'_id': 'Delta', 'negatif_sayisi': 955, 'ort_guven': 0.902, 'güven_seviyesi': 'yüksek'}
{'_id': 'Virgin America', 'negatif_sayisi': 181, 'ort_guven': 0.902, 'güven_seviyesi': 'yüksek'}
```

Şekil 17-18: Çok aşamalı pipeline ile negatif tweet güven seviyesi etiketleme — tüm havayolları "yüksek"

Sorgu 6 — Yüzdesel Dağılım (\$addFields + \$divide + \$multiply)

Şikayet sebeplerini gruplayıp her birinin toplam negatif tweetler içindeki yüzdesini hesaplıyorum. \$addFields ile pipeline ortasında yeni bir alan ekliyorum.

```
# once toplam negatif tweet sayisini aliyorum – yuzde hesabi icin lazim
# $addFields: mevcut dokumana yeni alan ekler (mevcut alanlara dokunmaz)
# $divide: bolme islemi – adet / toplam
# $multiply: carpma – bolum sonucunu 100 ile carparak yuzdeye ceviriyorum
# $round: 1 ondalik basamaga yuvarla
toplam_negatif = collection.count_documents({"airline_sentiment": "negative"})
print("Sorgu 6: Sikayet nedenlerinin yuzdeleri:")
pipeline3 = [
    {"$match": {"airline_sentiment": "negative", "negativereason": {"$ne": "Belirtilmedi"}}},
    {"$group": {"_id": "$negativereason", "adet": {"$sum": 1}}},
    {"$addFields": {
        "yuzde": {
            "$round": [{"$multiply": [{"$divide": ["$adet", toplam_negatif]}, 100]}, 1]
        }
    }},
    {"$sort": {"adet": -1}},
    {"$limit": 5}
]
for doc in collection.aggregate(pipeline3):
    print(doc)
```

```
Sorgu 6: Sikayet nedenlerinin yuzdeleri:
{'_id': 'Customer Service Issue', 'adet': 2902, 'yuzde': 31.7}
{'_id': 'Late Flight', 'adet': 1660, 'yuzde': 18.1}
{'_id': "Can't Tell", 'adet': 1190, 'yuzde': 13.0}
{'_id': 'Cancelled Flight', 'adet': 843, 'yuzde': 9.2}
{'_id': 'Lost Luggage', 'adet': 721, 'yuzde': 7.9}
```

Şekil 19-20: Şikayet nedenlerinin yüzde dağılımı — "Customer Service Issue" %31.7 ile birinci

Sorgu 7 — Bileşik İndeks (Compound Index) + explain()

Tekli indeks yerine compound (bileşik) indeks oluşturdum — airline ve airline_sentiment alanlarını birlikte indeksleyerek ikisini aynı anda filtreleyen sorguları hızlandırdım. explain() ile indeksin gerçekten kullanıldığını doğruladım. MongoDB bir sorgu yaptığında normalde tüm dokümanları tek tek tarar. İndeks varsa sadece ilgili dokümanlara gider (IXSCAN) — bu büyük veri setlerinde ciddi performans farkı yaratır.


```

# createIndex(): belirtilen alan(lar) için indeks oluşturur
# [("airline", 1), ("airline_sentiment", 1)]: bileşik indeks
# 1 = artan sıralama, -1 = azalan sıralama
# explain(): sorgunun nasıl çalıştığını gösterir – indeks kullandı mı kullanmadı mı
print("Sorgu 7: Bileşik indeks oluşturuluyor...")
collection.create_index([("airline", 1), ("airline_sentiment", 1)])

# indeksin kullanıldığını doğrula
aciklama = collection.find(
    {"airline": "Delta", "airline_sentiment": "negative"}
).explain()
kazanan_plan = aciklama['queryPlanner']['winningPlan']
if 'inputStage' in kazanan_plan:
    print("Kullanılan indeks:", kazanan_plan['inputStage'].get('indexName', 'bulunamadi'))
else:
    print("Plan detayı:", kazanan_plan)

```

✓ 0.0s

Sorgu 7: Bileşik indeks oluşturuluyor...
 Kullanılan indeks: airline_1_airline_sentiment_1

Şekil 21: Bileşik indeks oluşturma ve explain() ile indeksin kullanıldığının doğrulanması

Sorgu 8 — updateOne() + \$set ile Çoklu Alan Ekleme

En çok RT alan tweeti bulup özel olarak işaretliyorum. \$set ile birden fazla yeni alan ekliyorum.

```

# updateOne(): kosulu saglayan ilk dokumani gunceller
# $set: belirtilen alanlari ekler veya gunceller
# eger alan zaten varsa degerini degistirir, yoksa yeni alan olusturur
print("Sorgu 8: Viral tweet isaretleme:")
collection.update_one(
    {"retweet_count": {"$gt": 10}},
    {"$set": {
        "one_cikan": True,
        "kategori": "viral_tweet",
        "inceleme_notu": "yuksek etkilesim"
    }}
)
# guncellenen kaydi kontrol edelim
kontrol = collection.find_one(
    {"one_cikan": True},
    {"text": 1, "retweet_count": 1, "kategori": 1, "_id": 0}
)
print("Guncellenen kayıt:", kontrol)

```

Sorgu 8: Viral tweet isaretleme:
 Guncellenen kayıt: {'retweet_count': 22, 'text': '@SouthwestAir beautiful day in Seattle! <http://t.co/iqU0PPVq2S>', 'kategori': 'viral_tweet'}

Şekil 22-23: updateOne() + \$set ile viral tweet işaretleme ve doğrulama

Sorgu 9 — updateMany() + \$set + \$inc Birlikte Kullanım

Güven skoru tam 1.0 olan tüm tweetlere "kesin_sonuc: true" etiketi eklerken aynı zamanda retweet sayısını 1 artırıyorum. İki farklı güncelleme operatörünü tek sorguda birlikte kullanıyorum.

```
# updateMany(): kosulu saglayan TUM dokumanlari gunceller
# $set: yeni alan ekle
# $inc: mevcut sayisalalani belirtilen miktarda artir (increment)
# ikisini ayni anda yapabiliyorum – veritabanina 2 kere gitmeme gerek yok
print("Sorgu 9: Kesin sonuclari isaretle + RT artir:")
guncelleme = collection.update_many(
    {"airline_sentiment_confidence": 1.0},
    {
        "$set": {"kesin_sonuc": True},
        "$inc": {"retweet_count": 1}
    }
)
print(f"Guncellenen kayit sayisi: {guncelleme.modified_count}")
```

✓ 0.9s

Sorgu 9: Kesin sonuclari isaretle + RT artir:
Guncellenen kayit sayisi: 10406

Şekil 24: updateMany() + \$set + \$inc tek sorguda 10.406 kayıt güncelleme

Sorgu 10 — deleteMany() + Çoklu Koşul

Güven skoru 0.5'in altında VE sebebi "Can't Tell" olan kayıtları siliyorum. Bu kayıtlar düşük güven + belirsiz sebep taşıyor, yani "analist bile emin olamamış, sebebini de bilmiyoruz" durumunda. Bunları tutmak analize gürültü katar.

```
# deleteMany(): kosulu saglayan tum dokumanlari siler
# $lt: "less than" yani "kucuktur"
# $and ile iki kosulu birlikte kullaniyorum – ikisi de saglanmali
print("Sorgu 10: Dusuk guvenli + belirsiz kayitlari sil:")
silinen = collection.delete_many({
    "$and": [
        {"airline_sentiment_confidence": {"$lt": 0.5}},
        {"negativereason": "Can't Tell"}
    ]
})
print(f"Silinen kayit sayisi: {silinen.deleted_count}")
```

✓ 0.0s

Sorgu 10: Dusuk guvenli + belirsiz kayitlari sil:
Silinen kayit sayisi: 25

Şekil 25: deleteMany() + \$and ile düşük güven + belirsiz sebepli 25 kaydın silinmesi

Sorgu 11 — Regex + \$in Birleşik Kullanım

"cancel" veya "delay" kelimelerini tweet metninde arıyorum. Regex içinde pipe (|) operatörü ile iki kelimeyi tek seferde yakalıyorum. Ayrıca \$in ile sadece United ve Delta havayollarını filtreliyorum.

```
# $regex: metin icinde desen arama (regular expression)
# "cancel|delay": cancel VEYA delay kelimesini ara
# $options: "i" = case-insensitive (buyuk/kucuk harf farketmez)
# $in: belirtilen degerlerden herhangi birine esit mi kontrol et
print("Sorgu 11: cancel/delay iceren United/Delta tweetleri:")
sonuc11 = collection.find(
    {
        "text": {"$regex": "cancel|delay", "$options": "i"},
        "airline": {"$in": ["United", "Delta"]}
    },
    {"airline": 1, "text": 1, "negativereason": 1, "_id": 0}
).sort("airline_sentiment_confidence", -1).limit(3)
for i, doc in enumerate(sonuc11, 1):
    print(f"\n--- Sonuc {i} ---")
    print(f"  Havayolu : {doc['airline']}")
    print(f"  Sebep    : {doc['negativereason']}")
    print(f"  Tweet    : {doc['text']}")
```

Sorgu 11: cancel/delay iceren United/Delta tweetleri:

--- Sonuc 1 ---

Havayolu : Delta

Sebep : Cancelled Flight

Tweet : @JetBlue but by Cancelled Flighting my flight and pushing me to the next day I'd lose \$150 hotel which was why I was trying to get a same-day

--- Sonuc 2 ---

Havayolu : Delta

Sebep : Cancelled Flight

Tweet : @JetBlue Cancelled Flighted my flight. Went with another airline 2 leave 2day. They Cancelled Flighted also. Called JetBlue & got same fli

--- Sonuc 3 ---

Havayolu : Delta

Sebep : Late Flight

Tweet : @JetBlue why was Flight 1856 delayed to Buffalo ? Itâs a direct flight and the plane is at the gate.

Şekil 26-27: Regex + \$in birleşimi — Delta'nın iptal/gecikme tweetleri

Sorgu 12 — \$in + Aggregation Pipeline (Bileşik _id)

Büyük 3 havayolunu seçip her birinin duygu dağılımını (pozitif/negatif/nötr sayılarını) çıkardım. \$group içinde bileşik _id kullanarak havayolu + duygu çiftini grupluyorum.

```
# bileşik _id: grupta birden fazla alana göre yapılır
# {"havayolu": "$airline", "duygu": "$airline_sentiment"} şeklinde
# her havayolu-duygu çifti için ayrı bir grup oluşur
# örneğin: United-negative, United-positive, United-neutral
print("Sorgu 12: Büyük 3 havayolunun duygu dağılımı:")
pipeline_ileri = [
    {"$match": {"airline": {"$in": ["United", "American", "US Airways"]}}},
    {"$group": {
        "_id": {"havayolu": "$airline", "duygu": "$airline_sentiment"},
        "adet": {"$sum": 1}
    }},
    {"$sort": {"_id.havayolu": 1, "adet": -1}}
]
for doc in collection.aggregate(pipeline_ileri):
    print(doc)
```

```
Sorgu 12: Büyük 3 havayolunun duygu dağılımı:
{'_id': {'havayolu': 'American', 'duygu': 'negative'}, 'adet': 1939}
{'_id': {'havayolu': 'American', 'duygu': 'neutral'}, 'adet': 455}
{'_id': {'havayolu': 'American', 'duygu': 'positive'}, 'adet': 326}
{'_id': {'havayolu': 'US Airways', 'duygu': 'negative'}, 'adet': 2259}
{'_id': {'havayolu': 'US Airways', 'duygu': 'neutral'}, 'adet': 381}
{'_id': {'havayolu': 'US Airways', 'duygu': 'positive'}, 'adet': 269}
{'_id': {'havayolu': 'United', 'duygu': 'negative'}, 'adet': 2625}
{'_id': {'havayolu': 'United', 'duygu': 'neutral'}, 'adet': 697}
{'_id': {'havayolu': 'United', 'duygu': 'positive'}, 'adet': 492}
```

Şekil 28-29: Bileşik _id ile 3 havayolunun duygu dağılımı — US Airways en fazla negatif

Sorgu 13 — \$facet ile Çok Boyutlu Analiz

\$facet MongoDB'nin en güçlü pipeline operatörlerinden biri. Tek bir sorgu içinde birden fazla bağımsız pipeline çalıştırmayı sağlıyor. Ben burada aynı anda hem duygu dağılımını hem de en yaygın 3 şikayet sebebini hesaplıyorum — veritabanına tek bir sorgu göndererek.

```
# $facet: tek sorguda birden fazla pipeline calistir
# her alt-pipeline bagimsiz calisir ve sonuclari ayri ayri doner
# bu sayede veritabanina 1 kere gidip 2 farkli analiz sonucu aliyorum
# performans acisindan cok verimli - 2 ayri sorgu gondermekten iyi
print("Sorgu 13: Tek sorguda cok boyutlu analiz ($facet):")
facet_pipeline = [
    {"$facet": {
        "duygu_dagilimi": [
            {"$group": {"_id": "$airline_sentiment", "sayi": {"$sum": 1}}},
            {"$sort": {"sayi": -1}}
        ],
        "en_yaygin_sikayet": [
            {"$match": {"negativereason": {"$ne": "Belirtilmedi"}}},
            {"$group": {"_id": "$negativereason", "sayi": {"$sum": 1}}},
            {"$sort": {"sayi": -1}},
            {"$limit": 3}
        ]
    }}
]
sonuc = list(collection.aggregate(facet_pipeline))
print("Duygu dagilimi:", sonuc[0]['duygu_dagilimi'])
print("En yaygin 3 sikayet:", sonuc[0]['en_yaygin_sikayet'])
```

✓ 0.0s

Sorgu 13: Tek sorguda cok boyutlu analiz (\$facet):

Duygu dagilimi: [{"_id": "negative", "sayi": 9132}, {"_id": "neutral", "sayi": 3091}, {"_id": "positive", "sayi": 2353}]

En yaygin 3 sikayet: [{"_id": "Customer Service Issue", "sayi": 2902}, {"_id": "Late Flight", "sayi": 1660}, {"_id": "Can't Tell", "sayi": 1165}]

Şekil 30: \$facet ile tek sorguda hem duygu dağılımı hem de top 3 şikayet sebebi

\$facet ile tek bir aggregation sorgusunda hem genel duygu dağılımını hem de en yaygın 3 şikayet sebebini aynı anda hesapladım. Normalde bu 2 farklı bilgi için 2 ayrı sorgu yazmam gerekirdi. \$facet bunları tek sorguda birleştiriyor — özellikle büyük veri setlerinde performans kazancı sağlar.

Sorgu 14 — \$bucket ile Güven Skoru Aralık Analizi

Güven skorlarını belirli aralıklara bölüp her aralıktaki tweet sayısını hesaplıyorum. Bu bir tür histogram mantığı — MongoDB tarafında yapılması Python'da yapmaktan daha verimli.

```
# $bucket: sayisal bir alani belirtilen araliklara boler
# groupBy: hangi alana gore bolunecek
# boundaries: aralik sinirlari — [0, 0.4, 0.7, 0.9, 1.01] demek:
#   0-0.4 arasi, 0.4-0.7 arasi, 0.7-0.9 arasi, 0.9-1.01 arasi
# default: sinirlarin disinda kalan degerler icin
# output: her aralik icin ne hesaplanacak
print("Sorgu 14: Guven skoru aralik analizi ($bucket):")
bucket_pipeline = [
    {"$bucket": {
        "groupBy": "$airline_sentiment_confidence",
        "boundaries": [0, 0.4, 0.7, 0.9, 1.01],
        "default": "Diğer",
        "output": {
            "tweet_sayisi": {"$sum": 1},
            "ornek_havayolu": {"$first": "$airline"}
        }
    }}
]
for doc in collection.aggregate(bucket_pipeline):
    print(doc)
```

✓ 0.0s

```
Sorgu 14: Guven skoru aralik analizi ($bucket):
{'_id': 0, 'tweet_sayisi': 211, 'ornek_havayolu': 'Virgin America'}
{'_id': 0.4, 'tweet_sayisi': 3636, 'ornek_havayolu': 'Virgin America'}
{'_id': 0.7, 'tweet_sayisi': 310, 'ornek_havayolu': 'Virgin America'}
{'_id': 0.9, 'tweet_sayisi': 10419, 'ornek_havayolu': 'Virgin America'}
```

Şekil 31: \$bucket histogram — tweetlerin %70'i 0.9-1.0 güven aralığında

Güven skorlarını 4 aralığa böldüm (0-0.4: Çok düşük, 0.4-0.7: Düşük, 0.7-0.9: Orta, 0.9-1.0: Yüksek) ve her aralıktaki tweet sayısını hesapladım. Bu analiz verinin güven skoruna göre nasıl dağıldığını gösteriyor. Çoğu tweetin yüksek güvenle etiketlendiğini görüyorum — bu veri setinin kaliteli olduğunu gösterir.

Sorgu 15 — distinct() ile Benzersiz Değer Analizi

Her havayoluna kaç farklı şehirden tweet atıldığını bulmak için distinct kullanıyorum.

```
# distinct(): belirtilen alandaki benzersiz (tekil) degerleri dondurur
# SQL'deki SELECT DISTINCT karsiligi
# ornegin 14.000 tweette kac farkli sehir var?
print("Sorgu 15: Benzersiz deger analizi (distinct):")
benzersiz_konumlar = collection.distinct("tweet_location")
# "Bilinmiyor" olanlari cikariyorum
gercek_konumlar = [k for k in benzersiz_konumlar if k != "Bilinmiyor" and k is not None]
print(f"Toplam benzersiz konum sayisi: {len(gercek_konumlar)}")
print(f"Ilk 5 konum: {gercek_konumlar[:5]}")

# her havayoluna ait benzersiz sikayet sebebi sayisi
for havayolu in ["United", "Delta", "Southwest"]:
    sebepler = collection.distinct("negativereason", {"airline": havayolu})
    print(f"{havayolu}: {len(sebepler)} farkli sikayet sebebi")
```

```
Sorgu 15: Benzersiz deger analizi (distinct):
Toplam benzersiz konum sayisi: 3074
Ilk 5 konum: [' | | san antonio, texas|', ' Bronx, NY / Destin, Fl', ' California 92705', ' D(MD)V â\x9c\x88i, \x8f NYC â\x9c\x88i, \x8f Germany', ' DC | Je
United: 11 farkli sikayet sebebi
Delta: 11 farkli sikayet sebebi
Southwest: 11 farkli sikayet sebebi
```

Şekil 32-33: distinct() — 3074 benzersiz konum, havayollarında 11'er farklı şikayet

distinct() ile veri setindeki benzersiz konum sayısını ve her havayolunun kaç farklı şikayet sebebi aldığını hesapladım. Bu analiz veri setinin çeşitliliğini gösteriyor. Kaç farklı şehirden tweet atıldığını bilmek, verinin coğrafi kapsamını anlamamı sağlıyor.

AŞAMA 4: PYSPARK İLE DAĞITIK VERİ İŞLEME

Temizlenmiş veri setini PySpark DataFrame'e yükledim ve büyük veri işleme mantığıyla çeşitli operasyonlar uyguladım. Toplamda 10 işlem gerçekleştirdim.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, count, avg, round as spark_round, udf, length
from pyspark.sql.types import StringType
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# spark oturumunu baslatıyorum
# appName: Spark UI'da bu isimle görünür
# getOrCreate: zaten açık bir oturum varsa onu kullanır, yoksa yeni oluşturur
spark = SparkSession.builder \
    .appName('TwitterAnalysis') \
    .getOrCreate()

print("Spark başladı!")

dosya = '/content/drive/MyDrive/Buyuk_Veri_Donem_Projesi/twitter_big_data_pipeline/data/processed/cleaned_tweets.csv'

# csv yi pyspark dataframe olarak oku
# inferSchema=True: sütun tiplerini otomatik algılar (string, int, float vs.)
# quote/escape/multiline: tweet içindeki virgül ve satırbaşı sorunlarını onlar
spark_df = spark.read.csv(
    dosya,
    header=True,
    inferSchema=True,
    quote='\"',
    escape='\\',
    multiline=True
)

spark_df.printSchema()
```

```
Spark başladı!
root
|-- tweet_id: long (nullable = true)
|-- airline_sentiment: string (nullable = true)
|-- airline_sentiment_confidence: double (nullable = true)
|-- negativereason: string (nullable = true)
|-- negativereason_confidence: double (nullable = true)
|-- airline: string (nullable = true)
|-- airline_sentiment_gold: string (nullable = true)
|-- name: string (nullable = true)
|-- negativereason_gold: string (nullable = true)
|-- retweet_count: integer (nullable = true)
|-- text: string (nullable = true)
|-- tweet_coord: string (nullable = true)
|-- tweet_created: timestamp (nullable = true)
|-- tweet_location: string (nullable = true)
|-- user_timezone: string (nullable = true)
|-- cleaned_text: string (nullable = true)
```

Şekil 34-35: Spark başlatma ve printSchema() — 16 sütunun tipleri otomatik algılandı

4.1. Dönüşüm (Transformation) İşlemleri — 5 adet

Bu işlemler Spark'ın tembel değerlendirme prensibiyle çalışıyor. Yani bir transformation yazdığında Spark hemen çalıştırmıyor — sadece "şu işlem yapılacak" diye bir plan (DAG — Directed Acyclic Graph) oluşturuyor. Gerçek hesaplama ancak bir action (show, count, collect gibi) tetiklendiğinde başlıyor. Bu büyük verilerde çok önemli çünkü Spark planı optimize edebilir — mesela gereksiz adımları atlar veya işlemleri birleştirir.

İşlem 1 — filter + select (Karmaşık Filtreleme)

3 koşulu aynı anda uyguluyorum: Havayolu listesi + duygu + güven eşiği. isin() ile birden fazla değeri tek seferde kontrol edebiliyorum.

```
# filter(): satırları kosullara göre filtreler (SQL WHERE gibi)
# isin(): belirtilen degerlerden herhangi birine esit mi?
# col(): sutun adina referans verir
# &: VE (AND) operatoru - tum kosullar saglanmalı
# select(): sadece istedigim sutunlari sec (SQL SELECT gibi)
print("İslem 1: United/American negatif + guven>0.9:")
filtreli = spark_df.filter(
    (col("airline").isin("United", "American")) &
    (col("airline_sentiment") == "negative") &
    (col("airline_sentiment_confidence") > 0.9)
).select("airline", "negativereason", "airline_sentiment_confidence", "text")
filtreli.show(3, truncate=50)
```

İslem 1: United/American negatif + guven>0.9:

airline	negativereason	airline_sentiment_confidence	text
United	Cancelled Flight	1.0	@united still no refund or word via DM. Please ...
United	Late Flight	1.0	@united Delayed due to lack of crew and now del...
United	Flight Booking Problems	1.0	@united is the worst. Nonrefundable First class...

only showing top 3 rows

Şekil 36: filter + isin + select — United/American negatif, yüksek güvenli tweetler

İşlem 2 — withColumn + when/otherwise (Koşullu Sütun)

Güven skoruna göre kategori sütunu oluşturuyorum. Bu, SQL'deki CASE WHEN yapısının Spark karşılığı.

```
# withColumn(): mevcut DataFrame'e yeni sütun ekler
# when(): koşul sağlanırsa bu değeri ver
# otherwise(): hiçbir koşul sağlanmazsa bu değeri ver
# zincirleme when kullanabiliyorum – ilk sağlanan koşul geçerli olur
print("İşlem 2: Güven kategorisi sütunu:")
df_kategorili = spark_df.withColumn(
    "guven_kategori",
    when(col("airline_sentiment_confidence") == 1.0, "kesin")
    .when(col("airline_sentiment_confidence") >= 0.8, "yuksek")
    .otherwise("dusuk")
)
df_kategorili.select("airline", "airline_sentiment_confidence", "guven_kategori").show(5)
```

İşlem 2: Güven kategorisi sütunu:

airline	airline_sentiment_confidence	guven_kategori
Virgin America	1.0	kesin
Virgin America	0.3486	dusuk
Virgin America	0.6837	dusuk
Virgin America	1.0	kesin
Virgin America	1.0	kesin

only showing top 5 rows

Şekil 37: withColumn + when/otherwise — 3 kategorili güven etiketi

İşlem 3 — groupBy + Çoklu Aggregation

Havayollarına göre tweet sayısı, ortalama güven ve ortalama RT sayısını tek bir groupBy içinde hesaplıyorum. Veriyi tek geçişte işliyorum — 3 ayrı sorgu yazmaktan çok daha verimli.

```
# groupBy(): belirtilen sütuna göre grupla
# agg(): birden fazla aggregation fonksiyonunu aynı anda çalıştır
# alias(): sonuc sütununa isim ver
# orderBy(): sırala – desc() ile azalan
print("İşlem 3: Havayolu bazlı istatistikler:")
coklu_agg = spark_df.groupBy("airline").agg(
    count("*").alias("tweet_sayisi"),
    spark_round(avg("airline_sentiment_confidence"), 3).alias("ort_guven"),
    avg("retweet_count").alias("ort_rt")
).orderBy(col("tweet_sayisi").desc())
coklu_agg.show()
```

İşlem 3: Havayolu bazlı istatistikler:

airline	tweet_sayisi	ort_guven	ort_rt
United	3822	0.901	0.11015175300889586
US Airways	2913	0.922	0.0854788877445932
American	2723	0.916	0.0426000734484025
Southwest	2420	0.887	0.05991735537190083
Delta	2222	0.87	0.11341134113411341
Virgin America	504	0.876	0.051587301587301584

Şekil 38: groupBy + agg — havayolu bazlı istatistikler tek geçişte

İşlem 4 — UDF (User Defined Function) ile Tweet Uzunluk Kategorisi

Kendi fonksiyonumu yazıp bunu Spark DataFrame'ine uyguluyorum. UDF, Spark'ta kendi özel fonksiyonlarımızı tanımlamanın yolu. Tweet uzunluğuna göre "kısa", "orta", "uzun" kategorisi oluşturuyorum.

```
# UDF: User Defined Function – kendi yazdigim Python fonksiyonunu Spark'ta kullanmami saglar
# udf() ile fonksiyonu Spark'in anlayacagi formata ceviriyorum
# StringType(): fonksiyonun dondurdugu deger tipi (string)
# Neden UDF? Bazi islemler Spark'in hazır fonksiyonlariyla yapilamaz,
# bu durumda kendi fonksiyonumuzu yazariz

def tweet_uzunluk_kategorisi(text):
    """tweet uzunluguna gore kısa/orta/uzun etiketi ver"""
    if text is None:
        return "bilinmiyor"
    uzunluk = len(text)
    if uzunluk < 60:
        return "kısa"
    elif uzunluk < 120:
        return "orta"
    else:
        return "uzun"

# fonksiyonu UDF olarak kaydet
uzunluk_udf = udf(tweet_uzunluk_kategorisi, StringType())

# yeni sutun olustur
print("İşlem 4: Tweet uzunluk kategorisi (UDF):")
df_uzunluk = spark_df.withColumn("uzunluk_kat", uzunluk_udf(col("text")))
df_uzunluk.groupBy("uzunluk_kat").count().show()

İşlem 4: Tweet uzunluk kategorisi (UDF):
+-----+-----+
|uzunluk_kat|count|
+-----+-----+
|      orta| 5682|
|      kısa| 2209|
|      uzun| 6713|
+-----+-----+
```

Şekil 39: UDF ile tweet uzunluk kategorisi — çoğunluk "uzun"

İşlem 5 — Window Function ile Sıralama

Window function ile her havayolu içinde tweetleri güven skoruna göre sıralıyorum. Her tweet kendi havayolu grubu içinde kaçınıcı sırada olduğunu gösteriyor.

```
# Window: satirlari gruplar icinde siralamak veya hesaplama yapmak icin
# partitionBy: hangi alana gore grupla (her havayolu kendi icinde)
# orderBy: gruplari icinde neye gore sirala
# row_number(): her satira sirasina gore numara ver (1, 2, 3, ...)
# Boylece "her havayolunun en yuksek guvenli tweetleri" gibi sorulari cevaplayabilirim

from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

print("İşlem 5: Window function ile havayolu icinde siralama:")
pencere = Window.partitionBy("airline").orderBy(col("airline_sentiment_confidence").desc())

df_sirali = spark_df.withColumn("sira", row_number().over(pencere))

# her havayolunun en yuksek guvenli ilk 2 tweetini goster
df_sirali.filter(col("sira") <= 2) \
    .select("airline", "sira", "airline_sentiment_confidence", "airline_sentiment") \
    .orderBy("airline", "sira") \
    .show(12)
```

İşlem 5: Window function ile havayolu içinde sıralama:

airline	sıra	airline_sentiment_confidence	airline_sentiment
American	1	1.0	negative
American	2	1.0	negative
Delta	1	1.0	neutral
Delta	2	1.0	negative
Southwest	1	1.0	negative
Southwest	2	1.0	negative
US Airways	1	1.0	negative
US Airways	2	1.0	neutral
United	1	1.0	positive
United	2	1.0	positive
Virgin America	1	1.0	neutral
Virgin America	2	1.0	negative

Şekil 40-41: Window fonksiyonu — her havayolunun en güvenli 2 tweeti

Window fonksiyonu ile her havayolunu kendi içinde güven skoruna göre sıraladım ve her havayolunun en yüksek güvenli 2 tweetini gösterdim. Window function büyük veri işlemede çok güçlü bir araç. Normal groupBy grupları tek bir satıra indirger (ortalama, toplam gibi), ama window function her satırı koruyarak grupların içinde sıralama, kümülatif toplam gibi işlemler yapabilir.

4.2. Eylem (Action) İşlemleri — 2 adet

İşlem 6 — count + distinct (Farklı Sayım Türleri)

```
# count(): toplam satır sayısını verir — bu bir ACTION, yani hemen çalışır
# distinct(): benzersiz değerleri bul
# distinct().count(): benzersiz değerlerin sayısını ver
toplam = spark_df.count()
benzersiz_kullanici = spark_df.select("name").distinct().count()
benzersiz_konum = spark_df.filter(col("tweet_location") != "Bilinmiyor") \
    .select("tweet_location").distinct().count()
print(f"Toplam kayıt: {toplam}")
print(f"Benzersiz kullanıcı: {benzersiz_kullanici}")
print(f"Benzersiz konum (bilinmiyor hariç): {benzersiz_konum}")
```

Toplam kayıt: 14604
Benzersiz kullanıcı: 7701
Benzersiz konum (bilinmiyor hariç): 3081

Şekil 42: count + distinct — 14.604 kayıt, 7.701 kullanıcı, 3.081 konum

İşlem 7 — describe (İstatistiksel Özet)

```
# describe(): count, mean, stddev, min, max değerlerini hesaplar
# birden fazla sütun için aynı anda istatistik çıkarıyorum
print("İstatistiksel özet:")
spark_df.describe(['airline_sentiment_confidence', 'retweet_count']).show()
```

İstatistiksel özet:

summary	airline_sentiment_confidence	retweet_count
count	14604	14604
mean	0.899922760887427	0.08278553820870994
stddev	0.162954988530412	0.7466472933512464
min	0.335	0
max	1.0	44

Şekil 43: describe() — güven skoru ort. 0.90, RT dağılımı sola çarpık

4.3. Spark SQL — 2 adet

Spark DataFrame'i createOrReplaceTempView() ile geçici tabloya çevirip üzerinde standart SQL yazdım. Spark SQL, SQL bilen birisinin Spark'ı kolayca kullanmasını sağlar.

İşlem 8 — CASE WHEN + GROUP BY + HAVING

Her havayolunun duygu dağılımını ayrı sütunlarda gösteriyorum. HAVING ile sadece büyük havayollarını filtreliyorum.

```
# createOrReplaceTempView: Spark DataFrame'i SQL tablosu gibi kullanmami saglar
# CASE WHEN: kosullu degerlendirme - "eger sentiment negative ise 1 yaz, degilse 0 yaz"
# SUM(CASE WHEN...): kosulu saglayanlarin sayisini hesaplar
# HAVING: GROUP BY sonrasinda filtre uygular (WHERE gruplamadan ONCE, HAVING gruplamadan SONRA)
spark_df.createOrReplaceTempView('tweets_table')

print("SQL 1: Duygu dagilimi tablosu:")
sql1 = spark.sql("""
    SELECT
        airline,
        SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) as negatif,
        SUM(CASE WHEN airline_sentiment = 'positive' THEN 1 ELSE 0 END) as pozitif,
        SUM(CASE WHEN airline_sentiment = 'neutral' THEN 1 ELSE 0 END) as notr,
        COUNT(*) as toplam
    FROM tweets_table
    GROUP BY airline
    HAVING COUNT(*) > 500
    ORDER BY negatif DESC
""")
sql1.show()
```

```
SQL 1: Duygu dagilimi tablosu:
+-----+-----+-----+-----+-----+
| airline|negatif|pozitif|notr|toplam|
+-----+-----+-----+-----+-----+
| United|    2633|    492|   697|   3822|
| US Airways|    2263|    269|   381|   2913|
| American|    1941|    327|   455|   2723|
| Southwest|    1186|    570|   664|   2420|
| Delta|     955|    544|   723|   2222|
| Virgin America|    181|    152|   171|    504|
+-----+-----+-----+-----+-----+
```

Şekil 44-45: Spark SQL CASE WHEN — havayolu bazlı duygu tablosu, United en fazla negatif

İşlem 9 — Subquery (İç İçe Sorgu)

Önce her havayolunun negatif tweet oranını hesaplıyorum, sonra dış sorguda genel ortalamadan kötü olanları filtreliyorum. "Hangi havayolları genel ortalamadan daha kötü?" sorusunu tek SQL ile cevaplıyorum.

```
# Subquery: ic ice sorgu – bir sorgunun sonucunu baska bir sorguda kullan
# ic sorgu (alt_sorgu): her havayolunun negatif tweet yuzdesini hesaplar
# dis sorgu: ic sorgunun sonucunu filtreleyerek genel ortalamadan kotu olanlari buluyor
# WHERE ... > (SELECT ...): dis filtre icindeki subquery genel ortalamayi hesaplar
print("SQL 2: Ortalamanin ustunde negatif oranina sahip havayollari:")
sql2 = spark.sql("""
    SELECT
        airline,
        ROUND(negatif_oran, 2) as negatif_yuzde
    FROM (
        SELECT
            airline,
            (SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) as negatif_oran
        FROM tweets_table
        GROUP BY airline
    ) alt_sorgu
    WHERE negatif_oran > (
        SELECT SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
        FROM tweets_table
    )
    ORDER BY negatif_yuzde DESC
""")
sql2.show()
```

SQL 2: Ortalamanin ustunde negatif oranina sahip havayollari:

airline	negatif_yuzde
US Airways	77.69
American	71.28
United	68.89

Şekil 46: Subquery ile ortalama üstü negatif oran — US Airways, American, United

4.4. Pandas vs PySpark Karşılaştırması — 1 adet

İşlem 10 — Aynı İşlemi Pandas ve PySpark ile Yapıp Süre Karsılaştırması

```
import pandas as pd
import time

# aynı veriyi pandas ile de okuyalım
dosya='/content/drive/MyDrive/Buyuk_Veri_Donem_Projesi/twitter_big_data_pipeline/data/processed/cleaned_tweets.csv'
pandas_df = pd.read_csv(dosya)

# --- Pandas testi ---
# negatif tweetleri filtrele + havayoluna göre grupla + sayı ve ortalama hesapla
t1 = time.time()
pandas_sonuc = pandas_df[pandas_df['airline_sentiment'] == 'negative'].groupby('airline').agg(
    adet=('airline_sentiment', 'count'),
    ort_guven=('airline_sentiment_confidence', 'mean')
)
t2 = time.time()
pandas_sure = t2 - t1

# --- PySpark testi ---
# aynı işlem: filtrele + grupla + say + ortalama al + collect ile sonuc topla
t3 = time.time()
spark_sonuc = spark_df.filter(col("airline_sentiment") == "negative") \
    .groupBy("airline") \
    .agg(
        count("*").alias("adet"),
        avg("airline_sentiment_confidence").alias("ort_guven")
    ).collect()
t4 = time.time()
spark_sure = t4 - t3

print(f"Pandas suresi: {pandas_sure:.5f} sn")
print(f"PySpark suresi: {spark_sure:.5f} sn")

Pandas suresi: 0.01149 sn
PySpark suresi: 0.40985 sn
```

Şekil 47: Pandas vs PySpark hız karşılaştırması — bu ölçekte Pandas daha hızlı, Spark'ın avantajı büyük veri

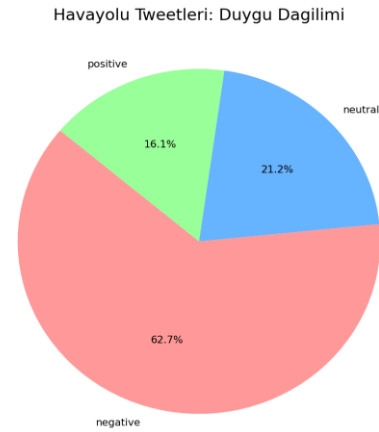
Pandas yaklaşık 0.01149 saniyede, PySpark yaklaşık 0.40985 saniyede bitti. Bu sonuç şaşırtıcı değil. Veri setimiz sadece ~14.000 satır ve tek makinede çalışıyoruz. PySpark burada Spark oturumunu başlatma, Directed Acyclic Graph oluşturma, görev dağıtımını gibi overhead'ler yüzünden yavaş kalıyor. Ama veri milyonlarca satıra çıktığında Spark dağıtık yapısıyla Pandas'ı çok geride bırakır. Küçük veride Pandas, büyük veride PySpark mantığı burada açıkça görülüyor.

ASAMA 5: VERİ GÖRSELLEŞTİRME VE ANALİZ

Matplotlib, Seaborn ve WordCloud kütüphaneleriyle 7 farklı grafik oluşturdum. Her birinin altında "bu grafik bize ne söylüyor?" sorusuna cevap verdim.

5.1. Duygu Dağılımı (Pie Chart)

```
# pasta grafik ile genel duygu dagilimi
# value_counts(): her kategoriden kac tane var hesaplar
# autopct: dilim icine yuzde yazar
plt.figure(figsize=(8, 8), dpi=70)
duygu_sayilari = df['airline_sentiment'].value_counts()
renkler = ['#ff9999', '#66b3ff', '#99ff99']
plt.pie(duygu_sayilari, labels=duygu_sayilari.index, autopct='%1.1f%%',
        startangle=140, colors=renkler)
plt.title('Havayolu Tweetleri: Duygu Dagilimi', fontsize=16)
plt.savefig('grafik1_duygu_dagilimi.png', dpi=150, bbox_inches='tight')
plt.show()
```



Şekil 48-49: Pasta grafik — negatif %62.7 ile baskın

Tweetlerin yaklaşık %63'ü negatif, %21'i nötr ve sadece %16'sı pozitif. İnsanlar havayoluyla iyi bir deneyim yaşadığında pek tweet atmıyor ama bir sorun yaşadığında hemen Twitter'a koşuyor. Bu, sosyal medya verilerinde sıkça karşılaşılan "negativity bias" durumu. Olumsuz deneyimler, olumlu deneyimlere kıyasla sosyal medyada çok daha fazla paylaşıyor.

5.2. Kelime Bulutu (WordCloud)

```
# tum temizlenmis tweet metinlerini tek bir string olarak birlestir
# WordCloud bu uzun stringdeki kelimelerin sikligina gore boyutlandirma yapar
tum_metin = " ".join(tweet for tweet in df['cleaned_text'].dropna())

plt.figure(figsize=(10, 6), dpi=70)
wc = WordCloud(width=800, height=400, background_color='white',
               max_words=100).generate(tum_metin)
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('Tweetlerde En Cok Gecen Kelimeler', fontsize=16)
plt.savefig('grafik2_wordcloud.png', dpi=150, bbox_inches='tight')
plt.show()
```



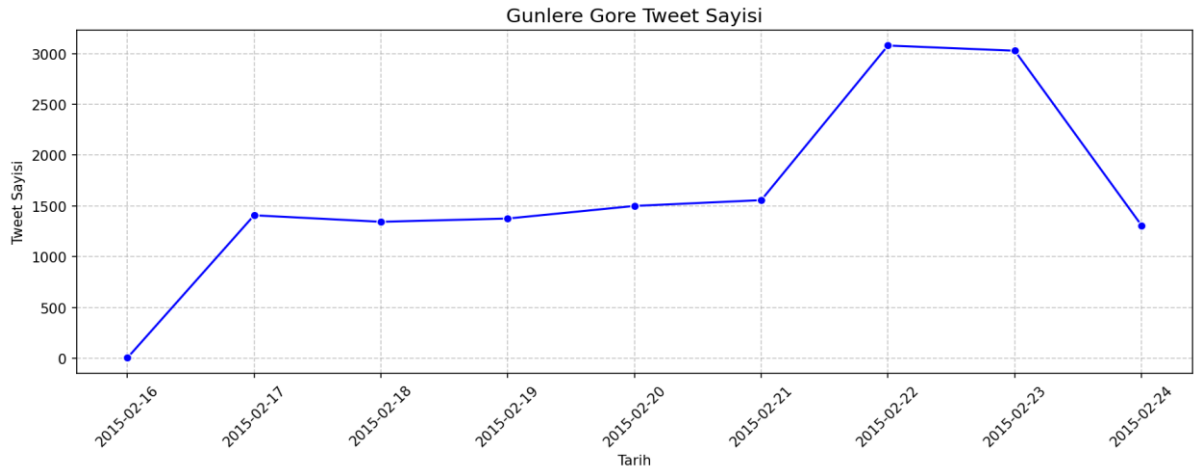
Şekil 50-51: WordCloud — "flight", "cancelled", "delay" öne çıkıyor

En büyük yazılan kelimeler "flight", "cancelled", "delay", "service" gibi operasyonel sorunlarla ilgili. Pozitif kelimeler (great, love, thank) çok küçük kalıyor. Bu da pasta grafikte tutarlı: İnsanlar genellikle uçuş iptallerinden ve rötarlardan şikayet ediyor. Kelime bulutu, metin madenciliği (text mining) açısından verinin ağırlık merkezini gösteriyor.

5.3. Zaman Serisi (Line Plot)

```
# gun bazli gruplama yaparak her gun kac tweet atilmis hesapliyorum
# dt.date ile datetime'dan sadece tarih kismini aliyorum
df['tarih_gun'] = df['tweet_created'].dt.date
gunluk = df.groupby('tarih_gun').size()

plt.figure(figsize=(12, 5), dpi=70)
sns.lineplot(x=gunluk.index, y=gunluk.values, marker='o', color='b')
plt.title('Gunlere Gore Tweet Sayisi', fontsize=14)
plt.xlabel('Tarih')
plt.ylabel('Tweet Sayisi')
plt.xticks(rotation=45)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('grafik3_zaman_serisi.png', dpi=150, bbox_inches='tight')
plt.show()
```

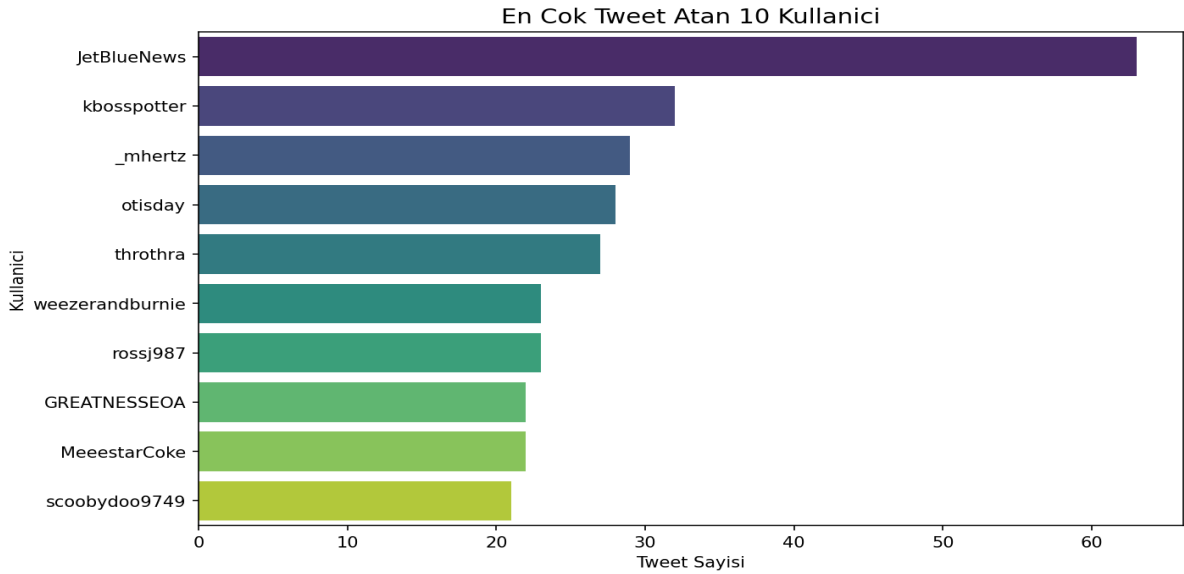


Şekil 52-53: Zaman serisi — kriz günlerinde ani tweet artışları

Belirli günlerde ani sıçramalar görüldüğü tespit edilmiştir. Bu zirve noktaları; büyük ihtimalle toplu uçuş iptallerinin veya olumsuz hava koşullarından kaynaklanan gecikmelerin yaşandığı günleri temsil etmektedir. Normal günlerde tweet sayısı görece sabit bir seyir izlerken, kriz günlerinde bir anda patlama yapmaktadır. Bir havayolu şirketi bu grafiği takip ederek hangi günlerde operasyonel sorunlar yaşadığını net bir şekilde tespit edebilir. Bu tür ani artışlar, veri madenciliğinde "anomali tespiti" bağlamında incelenmesi gereken oldukça önemli sinyallerdir.

5.4. En Aktif 10 Kullanıcı (Bar Plot)

```
# en çok tweet atan 10 kullanıcıyı bul ve bar grafik çiz
# value_counts(): her kullanıcının kaç tweet attığını sayar
# head(10): en fazla olan 10 tanesini al
plt.figure(figsize=(10, 6), dpi=70)
aktif = df['name'].value_counts().head(10)
sns.barplot(x=aktif.values, y=aktif.index, palette='viridis')
plt.title('En Çok Tweet Atan 10 Kullanıcı', fontsize=14)
plt.xlabel('Tweet Sayısı')
plt.ylabel('Kullanıcı')
plt.savefig('grafik4_aktif_kullanıcılar.png', dpi=150, bbox_inches='tight')
plt.show()
```



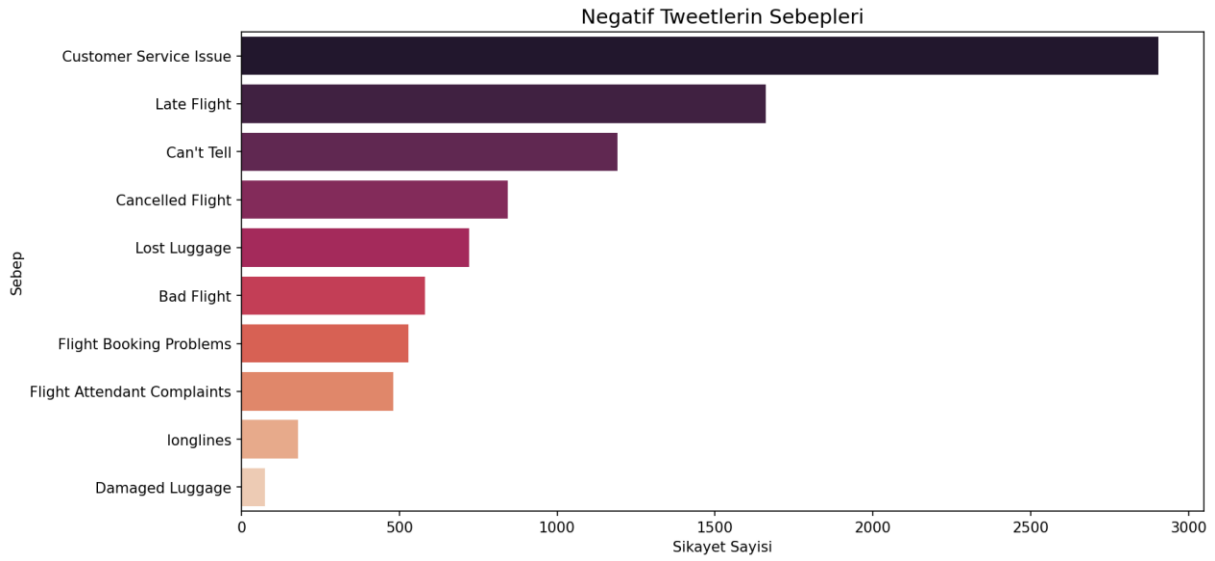
Şekil 54-55: En aktif 10 kullanıcı — JetBlueNews açık ara önde

Bazı kullanıcıların diğerlerinden çok daha fazla tweet attığı görülmektedir. Bu kişiler muhtemelen tek seferlik şikayetçiler değil; havayolu şirketiyle uzun süreli bir sorun yaşayan ve sürekli etkileşime giren kullanıcılardır. Bu "süper şikayetçi" profili, müşteri ilişkileri yönetimi (CRM) açısından öncelikli olarak ilgilenilmesi ve kriz yönetimi senaryolarına dahil edilmesi gereken kritik bir segmenttir.

5.5. Şikayetlerin Kök Nedenleri (Horizontal Bar)

```
# sadece negatif ve sebebi belli olanlari filtrele
# "Belirtilmedi" olanlari disarida birakiyorum cunku bunlar pozitif/notr tweetler
negatif = df[(df['airline_sentiment'] == 'negative') & (df['negativereason'] != 'Belirtilmedi')]

plt.figure(figsize=(12, 6), dpi=70)
sns.countplot(data=negatif, y='negativereason',
              order=negatif['negativereason'].value_counts().index,
              palette='rocket')
plt.title('Negatif Tweetlerin Sebepleri', fontsize=14)
plt.xlabel('Şikayet Sayisi')
plt.ylabel('Sebebi')
plt.savefig('grafik5_sikayet_nedenleri.png', dpi=150, bbox_inches='tight')
plt.show()
```



Şekil 56-57: Şikayet sebepleri — müşteri hizmetleri gecikmeden bile fazla

Müşteri memnuniyetsizliğinin bir numaralı sebebi "Customer Service Issues", yani müşteri hizmetleri sorunlarıdır. İkinci sırada ise "Late Flight" (rötar) yer alıyor. Burada ilginç olan nokta şu: İnsanlar uçağın rötar yapmasından ziyade, karşılarında muhatap bulamamaktan şikayet ediyor. Yani uçuş gecikse bile müşteri hizmetleri düzgün ve şeffaf bilgi verse, belki de bu kadar tepki oluşmayacak. Bir havayolu şirketi bu veriye bakarak, öncelikle müşteri hizmetleri kalitesini artırmaya odaklanmalıdır.

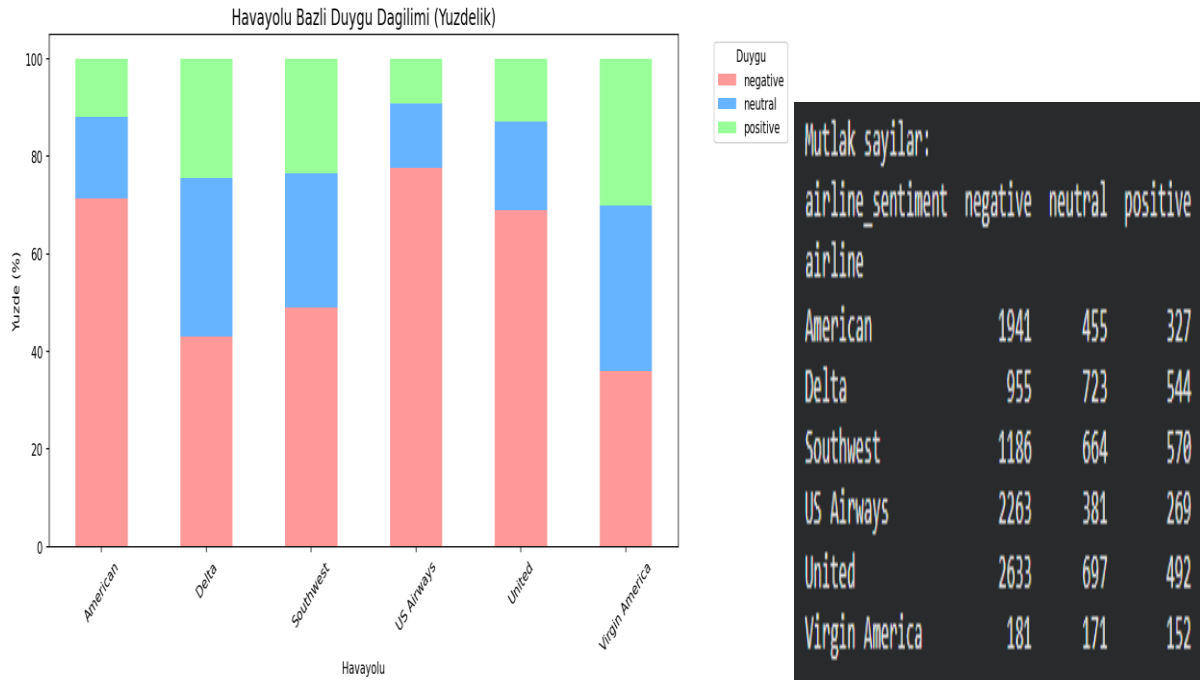
5.6. Havayolu Bazlı Duygu Karşılaştırması (Stacked Bar Chart)

```
# her havayolu için pozitif/negatif/notr tweet sayılarını hesapla
# crosstab: iki kategorik değişken arasındaki frekansları çıkarır
# normalize='index': satırlara göre yüzdelik hesaplar (her havayolunun toplamı %100)
ct = pd.crosstab(df['airline'], df['airline_sentiment'], normalize='index') * 100

# istersem mutlak sayıları da gösterebilirim
ct_mutlak = pd.crosstab(df['airline'], df['airline_sentiment'])

plt.figure(figsize=(12, 6), dpi=70)
ct.plot(kind='bar', stacked=True, color=['#ff9999', '#66b3ff', '#99ff99'],
        figsize=(12, 6))
plt.title('Havayolu Bazlı Duygu Dağılımı (Yüzdelik)', fontsize=14)
plt.xlabel('Havayolu')
plt.ylabel('Yüzde (%)')
plt.legend(title='Duygu', bbox_to_anchor=(1.05, 1))
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('grafik6_havayolu_uygu.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nMutlak sayılar:")
print(ct_mutlak)
```



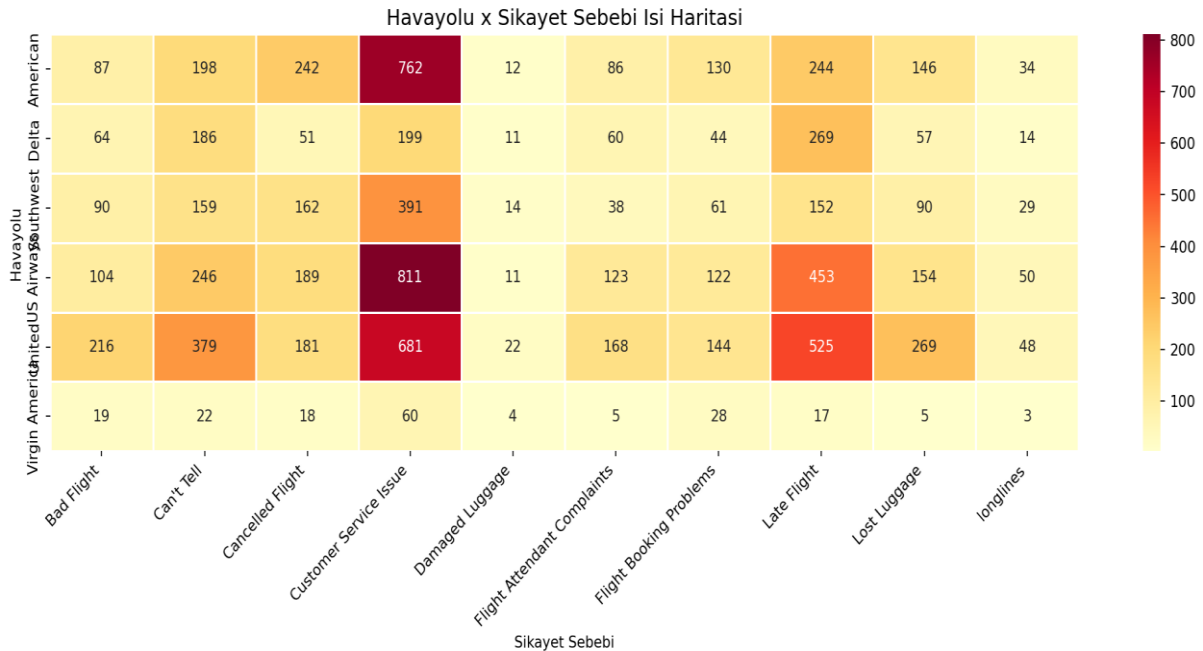
Şekil 58-59-60: Stacked bar — US Airways ve United en yüksek negatif oran

Bu grafik, hangi havayolunun nasıl bir duygu dağılımına sahip olduğunu doğrudan karşılaştırmayı sağlıyor. US Airways ve United en yüksek negatif duygu oranına sahipken; Virgin America ve Southwest nispeten daha iyi durumda görünüyor. Ancak burada dikkat edilmesi gereken bir nokta var: Virgin America'nın toplam tweet sayısı oldukça az. Bu yüzden oranları diğer büyük havayollarıyla doğrudan kıyaslamak yanıltıcı olabilir; çünkü küçük örneklem büyüklüğü istatistiksel güvenilirliği düşürür.

5.7. Şikayet Sebebi - Havayolu Isısı Haritası

```
# her havayolu için her şikayet sebebinden kaç adet var – crosstab ile hesapla
# sadece negatif tweetleri al ve sebebi belli olanları filtrele
negatif_df = df[(df['airline_sentiment'] == 'negative') & (df['negativereason'] != 'Belirtilmedi')]
ct_hm = pd.crosstab(negatif_df['airline'], negatif_df['negativereason'])

# heatmap: renk yoğunluğuyla sayıları görsel olarak gösterir
# annot=True: hücrelerin içine sayıları yazar
# cmap='YlOrRd': sarı-turuncu-kırmızı renk skalası (düşük=sarı, yüksek=kırmızı)
# fmt='d': sayıları tam sayı olarak göster (ondalık yok)
plt.figure(figsize=(14, 6), dpi=70)
sns.heatmap(ct_hm, annot=True, cmap='YlOrRd', fmt='d', linewidths=0.5)
plt.title('Havayolu x Şikayet Sebebi Isı Haritası', fontsize=14)
plt.xlabel('Şikayet Sebebi')
plt.ylabel('Havayolu')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('grafik7_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



Şekil 61-62: Isı haritası — havayolu × şikayet sebebi matrisi, United/US Airways "Customer Service"te koyu kırmızı

Bu ısı haritası, iki boyutlu bir analiz sunması yönüyle en güçlü görselleştirmelerden biridir. Her havayolunun hangi departmanda veya konuda daha çok şikayet aldığını tek bir bakışta görebiliyoruz. Örneğin; United Airlines'ın Müşteri Hizmetleri Sorunları hücrelerindeki koyu kırmızı renk, orada ciddi bir problem olduğuna işaret ediyor. American Airlines'ın Geciken Uçuşlar hücrelerindeki yoğunluk da oldukça dikkat çekici. Her havayolu şirketi, kendi tablosunda en koyu kırmızı olan hücreye öncelik vererek süreç iyileştirmesi yapabilir. Bu grafik, şirketler için bir nevi "şikayet önceliklendirme matrisi" görevi görmektedir.

AŞAMA 6: ETL DİYAGRAMI VE SİSTEM MİMARİSİ

6.1. ETL vs. ELT Karşılaştırması

ETL (Extract, Transform, Load): Veriyi kaynaktan çek, bir ara ortamda (bellekte veya başka bir sunucuda) temizle/dönüştür, ardından hedef veritabanına temiz haliyle yükle. Klasik veri ambarı yaklaşımıdır. Veri, hedefe ulaşmadan önce şekillenir. Genellikle veri kalitesinin kritik olduğu ve hedef sistemin işlem gücünün sınırlı kaldığı senaryolarda tercih edilir.

ELT (Extract, Load, Transform): Veriyi kaynaktan çek, olduğu gibi hedef sisteme yükle; temizleme ve dönüştürme işlemlerini hedef sistemin kendi işlem gücüyle orada gerçekleştirir. Bulut tabanlı veri gölü (data lake) yaklaşımlarında oldukça yaygındır. BigQuery ve Snowflake gibi modern sistemler, kendi içlerinde güçlü hesaplama kapasitelerine sahip oldukları için bu yaklaşıma oldukça uygundur.

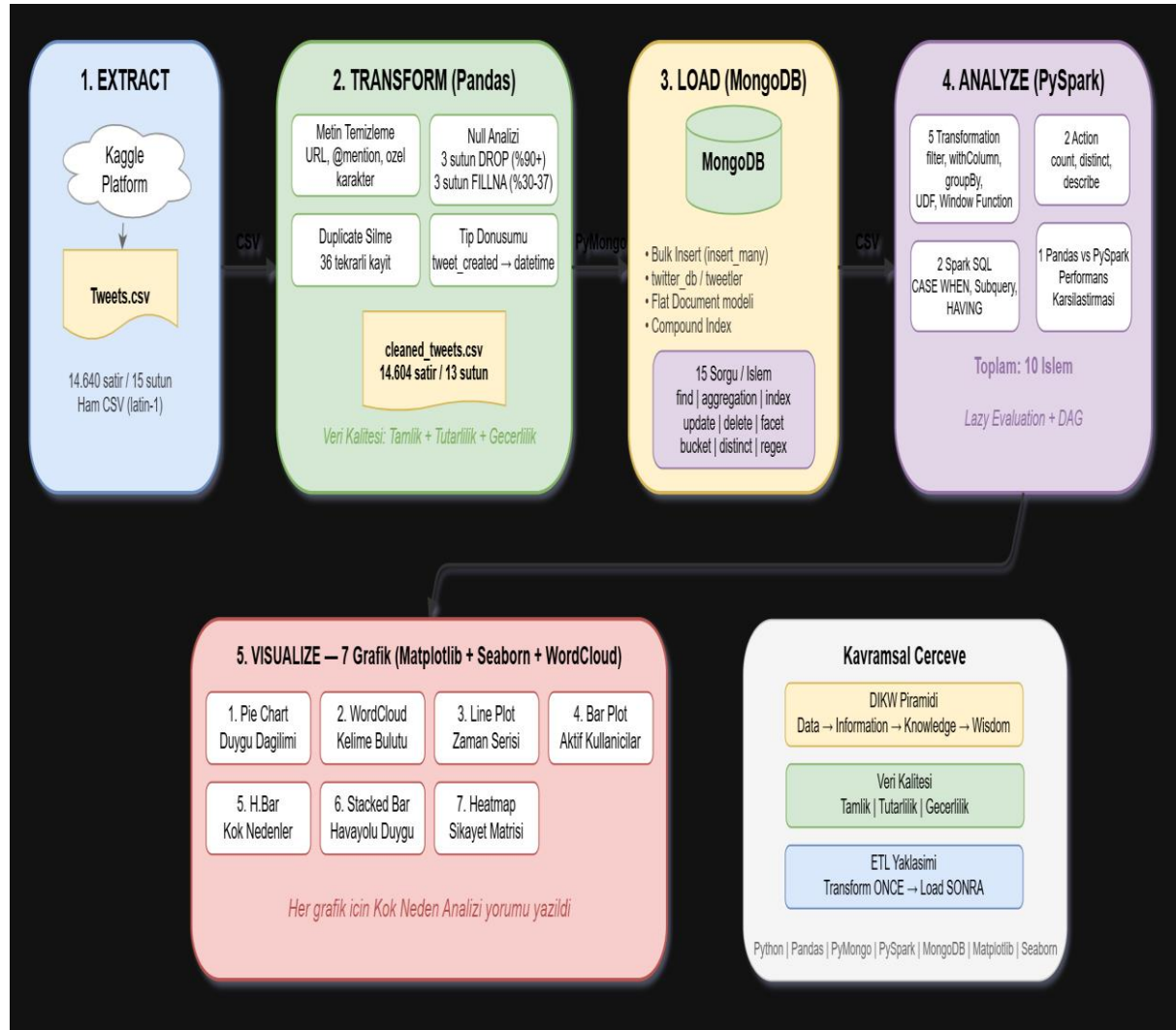
Benim bu projede uyguladığım akış kesinlikle **ETL** modeline daha yakındır. Süreç şu şekilde ilerlemiştir:

- **Extract (Veri Çıkarma):** Veriyi Kaggle'dan CSV formatında indirdim.
- **Transform (Veri Dönüştürme):** Veriyi doğrudan MongoDB'ye yüklemek yerine, öncelikle Python/Pandas ortamında temizledim. URL silme, eksik değer doldurma, gereksiz sütunları düşürme ve yinelenen (duplicate) kayıtları kaldırma gibi işlemleri bellekte (RAM) gerçekleştirdim.
- **Load (Veri Yükleme):** Veri tamamen temizlenip hazır hale geldikten sonra MongoDB'ye toplu yükleme (insert_many) yöntemiyle aktardım.

Eğer **ELT** yaklaşımını uygulasaydım; ham CSV dosyasını olduğu gibi MongoDB'ye yüklemem ve temizleme işlemlerini MongoDB'nin aggregation pipeline mimarisiyle veritabanı içinde yapmam gerekirdi. Ancak bu ölçekteki bir projede, Pandas ile temizlik yapmak hem daha kolay hem de daha kontrollü olduğu için ETL yaklaşımını tercih ettim.

6.2. Uçtan Uca Proje Akış Diyagramı

Aşağıdaki diyagram projenin tüm aşamalarını göstermektedir:



Şekil 63: ETL akış diyagramı — 6 aşamalı pipeline özeti

SONUÇ

Bu projede, Kaggle'dan aldığım havayolu tweet verisini baştan sona işledim. Veri temizleme aşamasında; yüksek null oranına sahip 3 sütunu düşürdüm, orta düzeyde null oranı içeren 3 sütunu anlamlı değerlerle doldurdum, 36 tekrarlı kaydı sildim, metin temizleme ve tip dönüşümlerini gerçekleştirdim. MongoDB tarafında 15 farklı sorgu ve işlem; PySpark ile ise 10 farklı dağıtık işlem uyguladım. Görselleştirme adımında 7 farklı grafik oluşturarak her birinin kök neden analizini yaptım. Proje boyunca derste öğrendiğimiz; DIKW piramidi, veri kalitesi boyutları, NoSQL modelleme, dağıtık işleme prensipleri, ETL kavramı ve kök neden analizini doğrudan pratiğe dökmüş oldum. En dikkat çekici bulgularım ise şunlar oldu: Havayolu müşterilerinin en çok müşteri hizmetleri konusunda şikayet etmesi, tweet trafiğinin kriz günlerinde patlama yapması, veri setindeki güçlü negatif yanlılık ve ısı haritası analiziyle her havayolunun kendine özgü bir şikayet örüntüsüne sahip olduğunu keşfetmem.